

Introduction to Python

Author: Refat Othman

1. What is Programming & What is an Algorithm?

Programming = giving a computer clear steps to follow.

An **algorithm** is like a recipe.

Real-Life Example: Algorithm for Making Tea

1. Boil water
2. Put teabag in cup
3. Pour hot water
4. Wait 3 minutes
5. Add sugar/milk
6. Serve

Example: Simple Step-by-Step Program

```
print("Boiling water...")  
print("Adding teabag...")  
print("Tea is ready!")
```

This shows how a computer follows steps exactly.

2. Python Basics

Python is easy to read and widely used in AI.

Example: Your First Python Program

```
print("Hello, world!")
```

Variables Example

```
name = "Ali"      # string
year = 2025       # integer
height = 1.75     # float
print(name, year, height)
```

Input/Output Example

```
name = input("Enter your name: ")
print("Nice to meet you,", name)
```

Basic Calculation Example

```
a = float(input("Enter a number: "))
b = float(input("Enter another number: "))
print("Their sum is:", a + b)
```

3. Conditions (If Statements)

Conditions let the program choose a path.

Example: Checking Grades

```
score = int(input("Enter your score: "))

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("C or below")
```

Example: Login Check

```
username = input("Username: ")
password = input("Password: ")

if username == "admin" and password == "1234":
    print("Login successful")
else:
    print("Access denied")
```

4. Loops (For & While)

Loops repeat tasks.

For Loop Example (Counting)

```
for i in range(1, 6):  
    print("Step", i)
```

While Loop Example (Password Retry)

```
password = ""  
while password != "1234":  
    password = input("Enter password: ")  
print("Correct! Access granted.")
```

Example: Sum of Numbers

```
total = 0  
for i in range(1, 6):  
    total += i  
print("Sum of 1 to 5 is", total)
```

5. Lists (Arrays), Sets, and Hashes (Dictionaries)

In Python, we often store groups of data in three main structures: **lists**, **sets**, and **dictionaries (hashes)**. Below you will find **detailed explanations**, **beginner-friendly examples**, and a **final comparison table**.

5.1 Lists (Arrays)

A **list** is an ordered collection of items. This means:

- Items keep the order you insert them in.
- You can access items by position (index).
- Lists **allow duplicates**.
- Lists are **mutable** → you can change them.
- Best used when order matters.

Real-Life Analogy

A list is like a **shopping list**: apples, milk, bread. You know the exact order—and you can add/remove items.

Example: Creating and Using a List

```
names = ["Ali", "Sara", "John"]
print(names)           # ['Ali', 'Sara', 'John']
print(names[1])        # Sara (access by index)
```

Updating a List

```
names.append("Mona")    # Add a new item
names[0] = "Ahmad"      # Modify existing item
```

Duplicates Allowed

```
numbers = [10, 20, 20, 30]
print(numbers) # [10, 20, 20, 30]
```

When to use a list?

- Keep items in order
- Need duplicates
- Want to edit values (add/remove/change)

5.2 Sets

A **set** is an *unordered* collection of items. It:

- Has **no duplicates** → automatically removes repeated values
- Has **no indexing** → cannot use numbers like list[0]
- Is very fast for checking if something exists

Real-Life Analogy

A set is like a **collection of unique stickers**: If you add the same sticker twice, you still have only one.

Example: Creating a Set

```
unique_nums = {10, 20, 20, 30}
print(unique_nums) # {10, 20, 30}
```

Duplicates automatically disappear.

Membership Check

```
print(20 in unique_nums) # True
print(40 in unique_nums) # False
```

When to use a set?

- You want **unique values** only
- You don't care about order
- You need **fast checking** (e.g., AI category labels)

5.3 Dictionaries (Hashes)

A **dictionary** stores data in **key** → **value pairs**.

- Keys must be unique
- Values can be anything (even lists)
- Fast access using the key, not index

Real-Life Analogy

A dictionary is like your **phone contacts**:

- Name = key
- Phone number = value

You search by name, not by position.

Example: Creating a Dictionary

```
student = {
    "name": "Ali",
    "age": 20,
    "grade": "A"
}

print(student["name"]) # Ali
print(student["grade"]) # A
```

Adding or Changing Values

```
student["age"] = 21 # modify value
student["city"] = "Bolzano" # add new key-value
```

Dictionary with List Values

Great for AI labels or JSON-like structures.

```
data = {
    "labels": ["spam", "ham", "urgent"],
    "accuracy": 0.92
}
```

When to use a dictionary?

- You need **fast lookup** by a name/key
- Data is naturally paired (like ID→Name, Word→Meaning)
- You want structured data like JSON

5.4 Summary Comparison (Beginner-Friendly)

Feature	List (Array)	Set	Dictionary (Hash)
Structure type	Ordered collection	Unordered unique collection	Key → Value pairs
Keeps order?	Yes	✗ No	✗ No (but keeps insertion order)
Allows duplicates?	Yes	✗ No	Keys ✗ No, Values Yes
How to access items?	By index (0,1,2...)	Cannot access by index	By key (e.g., student["age"])
Editable?	Yes	Yes	Yes
Best used for	Ordered data	Unique values	Fast lookup, structured data

6. Functions

Functions help you reuse code.

Example: Function Without Parameters

```
def greet():
    print("Hello!")

greet()
```

Example: Function With Parameters

```
def add(a, b):  
    return a + b  
  
print(add(3, 5))
```

Example: Function That Calls Another Function

```
def square(x):  
    return x * x  
  
def show_square(n):  
    print(square(n))  
  
show_square(4)
```

Recursive Example (Factorial)

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5))
```

7. File Handling

Write to a File

```
with open("data.txt", "w") as f:  
    f.write("Hello! This is saved in a file.\n")
```

Read from a File

```
with open("data.txt", "r") as f:  
    content = f.read()  
    print(content)
```

8. Introduction to Object-Oriented Programming (OOP)

OOP lets us group data + behavior.

Simple Class Example

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(self.name, "says Woof!")

mydog = Dog("Rex")
mydog.bark()
```

Real-Life Analogy

- A **class** = blueprint for building a car
- An **object** = actual car you can drive

9. Inheritance & Polymorphism

Inheritance Example

```
class Person:
    def greet(self):
        print("Hello!")

class Student(Person):
    pass

s = Student()
s.greet()
```

Student reuses Person's code.

Polymorphism Example

Same method name → different behavior.


```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        print("Bark")

class Cat(Animal):
    def speak(self):
        print("Meow")

for a in [Dog(), Cat()]:
    a.speak()
```

10. Abstraction, Interfaces & Exceptions

Abstraction Example

Hide details from users.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Exception Example

Protect program from crashing.

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
    print("Please enter a valid number.")
```

11. Detailed Comparison of OOP Class Concepts

11.1 Class vs Object

What is a Class?

A **class** is a design, plan, or blueprint. It does NOT exist in the real world—it's an idea.

Examples of a class:

- A blueprint of a house
- A recipe for a cake
- A plan for a car

What is an Object?

An **object** is something real created from the blueprint.

Examples of an object:

- The actual house built from the blueprint
- The cake cooked using the recipe
- A real car built from the plan

Code Example

```
class Dog:          # class = blueprint
    def bark(self):
        print("Woof!")

mydog = Dog()       # object = real thing created
mydog.bark()
```

11.2 Inheritance (Reusing Code)

Inheritance allows one class (child) to reuse code from another class (parent).

Real-Life Example:

- A "Student" is a type of "Person"
- A "Dog" is a type of "Animal"

Code Example

```
class Person:
    def greet(self):
        print("Hello!")

class Student(Person):
    pass
```

```
s = Student()
s.greet()    # Student uses Person's method
```

11.3 Polymorphism (Same Word, Different Action)

Polymorphism means **many forms**. Same method name → different behavior.

Real-Life Examples:

- The word "run" can mean running fast, running a company, running a program

Code Example

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        print("Bark")

class Cat(Animal):
    def speak(self):
        print("Meow")
```

11.4 Encapsulation (Protecting Data)

Encapsulation = wrapping data + methods and protecting them.

Real-Life Examples:

- Your bank card PIN is hidden
- Your phone has apps inside but you can't see the internal code

Code Example

```
class BankAccount:
    def __init__(self):
        self.__pin = "1234"    # private
```

11.5 Abstraction (Hiding Complexity)

Abstraction = showing only what the user needs, hiding how it works.

Real-Life Examples:

- You press "withdraw" at an ATM; you don't see circuits or code
- Driving a car: you turn a key, engine starts (you don't see details)

Code Example

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

11.6 Full Comparison Table of OOP Concepts

Below is an expanded, beginner-friendly comparison including **attributes vs methods**, **types of variables**, and how each OOP concept works.

Key Terms Before the Table

- **Attribute** → A variable inside a class (example: `self.name`)
- **Method** → A function inside a class (example: `def bark(self):`)
- **Instance Variable** → Belongs to the object; unique for each object
- **Class Variable** → Shared across all objects of the class
- **Method Types** → instance methods, class methods, static methods

Full OOP Comparison Table (Detailed)

Concept	Simple Definition	Real-Life Analogy	Code Meaning	Attributes	Methods
Class	A blueprint or template	A cake recipe	Defines attributes + methods	Declared inside the class	Declared inside the class
Object	A real instance created from a class	The actual cake	Uses class properties	Has its own values for attributes	Can call class methods
Inheritance	Reusing code from a parent class	A student inherits traits from a person	Child class gets parent attributes & methods	Inherits attributes + can add new ones	Inherits methods + can override them

Concept	Simple Definition	Real-Life Analogy	Code Meaning	Attributes	Methods
Polymorphism	Same method name, different behavior	"Run" has many meanings	Method overriding in child classes	Attribute use stays same	Methods behave differently in each class
Encapsulation	Protecting data	Hiding a PIN	Uses private variables (<code>__pin</code>)	Protects attributes using naming rules	Accessed via getter/setter methods
Abstraction	Hiding complexity	ATM hides internal mechanics	Shows only essential methods	Attributes hidden from user	Methods defined but implemented in child classes

Extra Section: Types of Variables in a Class

Students must understand the three types clearly.

1. Instance Variables (most common)

- Defined with `self`
- Unique for each object

```
class Car:
    def __init__(self, model):
        self.model = model      # instance variable
```

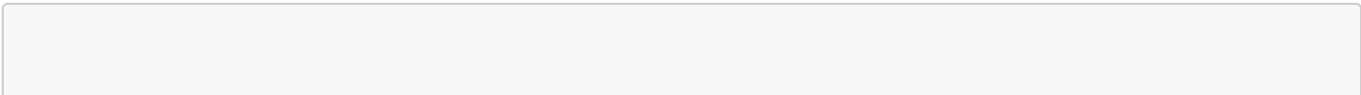
2. Class Variables (shared by all objects)

- Defined outside methods
- Same for all objects

```
class Car:
    wheels = 4      # class variable shared by all cars
```

3. Local Variables

- Defined inside a method
- Exist only while the method is running



```
def func():  
    x = 10 # local variable
```

Extra Section: Types of Methods

1. Instance Method (most common)

Uses `self` to access object data.

```
def drive(self):  
    print(self.model)
```

2. Class Method

Uses `@classmethod` and `cls`.

```
@classmethod  
def info(cls):  
    print("All cars have wheels =", cls.wheels)
```

3. Static Method

No access to `self` or `cls`.

```
@staticmethod  
def welcome():  
    print("Welcome to the Car class!")
```

12. Python Interview Questions

These help students prepare for job interviews.

- 1 What is the difference between a class and an object?
- 2 What is the purpose of the `__init__` method in Python?
- 3 What is inheritance? Give a simple example.
- 4 What is polymorphism and how does Python support it?
- 5 What is encapsulation and why is it important?

- 6 What is abstraction? Provide a real-life analogy.
 - 7 What are the differences between a list, set, and dictionary?
 - 8 What is the difference between `==` and `is` in Python?
 - 9 Why do we use `try/except` in Python? What problem does it solve?
 - 10 What is the difference between a function and a method?
-